

第四章 可执行文件的生成与加载执行

一、名词解释

链接：在计算机编程中，链接是指将多个源文件或目标文件中的代码和数据组合成一个可执行文件或共享库文件的过程。

可重定位文件：包含了程序的代码和数据，但是还没有完成地址的最终分配，可以在链接时进行地址的重定位，得到可执行文件。

可执行文件：经过链接之后，完成了地址的最终分配，可以直接在操作系统上执行的文件。

符号解析：在链接过程中，将符号（如函数名、变量名）与其在程序中的地址或者其他符号相关联的过程。

重定位：将程序中的符号引用与其真正的地址或者其他符号关联起来的过程。

ELF 头：Executable and Linkable Format 头，是一种用于描述可执行文件和共享库文件结构的数据结构。

节头表：包含了可执行文件或共享库文件中各个节（**section**）的信息，如起始地址、大小等。

程序头表：包含了可执行文件或共享库文件中各个程序段（**segment**）的信息，如加载地址、大小等。

只读代码段：存放程序的代码，只允许读取操作，不允许写入操作。

可读/写数据段：存放程序的数据，允许读取和写入操作。

全局符号：在整个程序中都可见的符号，可以被其他文件引用。

外部符号：在当前文件中未定义，但在其他文件中定义的符号。

本地符号：仅在当前文件中可见的符号，对其他文件不可见。

强符号：在链接过程中，优先选择的符号。

COMMON 符号：未被初始化的全局变量或未定义的外部变量的符号。

静态链接：在编译时将所有的库文件链接到可执行文件中的链接方式。

共享库文件：包含了可执行文件所需的代码和数据，可以在多个进程间共享使用的文件。

动态链接：在程序运行时将共享库文件链接到内存中的链接方式。

动态链接器：负责在程序运行时将共享库文件链接到内存中的系统程序。

动态链接库：包含了可执行文件所需的代码和数据，可以在程序运行时被动态链接到内存中使用的库文件。

进程：计算机中正在运行的程序的实例，是操作系统进行资源分配和调度的基本单位。

进程描述符：操作系统中用于描述进程的数据结构。

命令行解释程序：接收用户输入的命令行，并将其解释为操作系统能够执行的命令的程序。

指令周期：CPU 执行一条指令所经历的时间周期。

指令译码器：负责将指令解码成对应的操作控制信号的硬件模块。

内部异常：由程序执行过程中的错误或不正常情况引起的异常。

外部中断：由外部设备（如硬件中断、时钟中断）引起的中断。

数据通路：CPU 中负责数据传输和处理的部件的集合。

执行部件：CPU 中负责执行指令的部件，包括算术逻辑单元(ALU)等。

功能部件：CPU 中用于实现特定功能的部件，如乘法器、除法等。

控制器：CPU 中负责控制整个运算过程的部件。

时钟信号：CPU 内部的时钟信号，用于同步各个部件的工作。

指令流水线：CPU 中用于提高指令执行速度的一种技术，将指令执行过程划分为多个阶段并行处理。

二、简答题

(1)如何将多个 C 程序模块组合生成可执行文件?简述从源程序到可执行文件的转换过程。

将多个 C 程序模块组合生成可执行文件的过程主要包括编译（**Compilation**）、汇编（**Assembly**）、链接（**Linking**）三个阶段。

1. 编译阶段：对每个 C 程序模块进行编译，生成对应的目标文件（**Object File**），目标文件中包含了汇编代码但还没有进行地址分配。编译器将源代码转换成汇编代码，并生成目标文件。

2. 汇编阶段：对每个目标文件进行汇编，生成对应的目标文件的汇编文件（**Assembly File**），汇编器将汇编代码转换成机器语言代码，生成可重定位文件（**Relocatable File**）。

3. 链接阶段：将所有的可重定位文件以及所需的库文件进行链接，生成最终的可执行文件。链接器会将各个模块的目标文件组合起来，解析符号引用并进行地址重定位，最终生成一个完整的可执行文件。

简而言之，从源程序到可执行文件的转换过程包括编译、汇编和链接三个阶段，分别将源代码转换为目标文件、汇编文件和最终的可执行文件。在链接阶段，通过解析符号引用和地址重定位，将各个模块组合成一个完整的可执行文件。

(2)可重定位目标文件和可执行目标文件的主要差别是什么？

可重定位目标文件是经过编译和汇编，但尚未进行链接的文件，其中的地址还没有确定。

可执行目标文件是经过编译、汇编和链接，地址已经最终确定，可以直接在操作系统上执行的文件。

(3)链接器主要完成哪两方面的工作？

符号解析：解析各个模块之间的符号引用，确定符号对应的地址。

重定位：将各个模块的地址关联起来，生成最终的可执行文件。

(4)可重定位文件的.text 节、.rodata 节、.data 节和.bss 节中分别主要包含什么信息？

.text 节：存放程序的代码。

.rodata 节：存放只读数据，如常量字符串等。

.data 节：存放已初始化的全局和静态变量。

.bss 节：存放未初始化的全局和静态变量。

(5)可执行目标中有哪两种可装入段？哪些节组合成只读代码段？哪些节组合成可读/写数据段？

只读代码段：由.text 节和.rodata 节组合而成。

可读/写数据段：由.data 节和.bss 节组合而成。

(6)加载可执行文件时，加载器根据其中哪个表的信息对可装入段进行映射？

加载器根据程序头表（Program Header Table）中的信息对可装入段进行映射。

(7)静态链接和动态链接的差别是什么？

静态链接将所有的库文件链接到可执行文件中，在编译时就已经确定了所有的符号引用；动态链接是在程序运行时将共享库文件链接到内存中，允许多个程序共享同一个库文件，需要动态链接器在运行时解析符号引用。

(8)在可执行文件中将可装入段映射到虚拟地址空间，以形成每个进程独立的虚拟地址空间，这种做法有什么好处？

将可装入段映射到虚拟地址空间可以形成每个进程独立的虚拟地址空间，避免了进程间的地址冲突，提高了系统的安全性和稳定性。

(9)简述通过 shell 命令行解释程序进行程序加载的过程。

通过 shell 命令行解释程序进行程序加载的过程大致包括以下几个步骤：

用户输入命令行。

shell 解释命令行，识别要执行的程序。

shell 创建一个新的进程来执行该程序。

加载器加载可执行文件到内存中。

加载器执行程序入口点。

(10)CPU 的基本组成和基本功能各是什么？

CPU 的基本组成包括算术逻辑单元（ALU）、控制单元（Control Unit）、寄存器组。其基本功能是执行指令、进行算术和逻辑运算、控制程序的执行流程。

(11)如何控制一条指令执行结束后能够接着另一条指令执行？

控制一条指令执行结束后能够接着另一条指令执行是通过程序计数器（PC）的自增实现的，每执行一条指令，PC 自动加一，指向下一条指令的地址。

(12)通常一条指令的执行要经过哪些步骤？每条指令的执行步骤都一样吗？

一条指令的执行通常经过取指令、译码、执行、访存、写回这几个步骤。每条指令的执行步骤并不完全相同，不同的指令可能会有不同的执行路径和需要访问的数据。

(13)流水线方式下，一条指令的执行时间缩短了还是加长了？程序的执行时间缩短了还是加长了？

在流水线方式下，一条指令的执行时间缩短了，因为多条指令可以并行执行；但程序的执行时间可能会加长，因为流水线的阶段之间可能存在数据相关性或者控制相关性，导致流水线停顿或者冒险，从而影响程序的执行效率。

(14)具有什么特征的指令系统易于实现指令流水线？

易于实现指令流水线的特征包括：指令长度固定、指令之间无数据相关性、指令之间无控制相关性、指令之间的执行步骤相对简单等。